JAVA TECHNICAL TRAINING PRODUCT BATCH-I
SESSION:04 DATE:02/03/2026

1.LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS (LEETCODE PROBLEM NO:03)

I) BRUTE FORCE APPROACH
O(n^2)-Time Complexity

```java
import java.util.*;
public class A04 {
   public static void main(String args[])
   {
       String s="abcabcbb";
       int maxlength=-1;
       HashMap<Character,Integer> h1=new HashMap<>();
       for(int i=0;i<s.length();i++)
       {
           h1.clear();
           for(int j=i;j<s.length();j++)
           {
               if(h1.containsKey(s.charAt(j)))
               {
                   break;
               }
               h1.put(s.charAt(j),1);
               maxlength=Math.max(maxlength,j-i+1);
           }
       }
       System.out.println(maxlength);
   }
}
```

```
Run    A04
C:\Users\cse&student\.jdks\openjdk-25.0.2\bin\java.exe "-javaager
3

Process finished with exit code 0
```

2) OPTIMAL APPROACH

O(n)-Time Complexity

```java
import java.util.*;
public class A04 {
   public static void main(String args[])
   {
       String s="abcabcbb";
       int maxlength=-1;
       int left=0,right=0;
       HashMap<Character,Integer> h1=new HashMap<>();
       while(right<s.length())
       {
           while(left<right && h1.containsKey(s.charAt(right)))
           {
               left++;
               h1.remove(s.charAt(left));
           }
           maxlength=Math.max(maxlength,right-left+1);
           h1.put(s.charAt(right),1);
           right++;
       }
       System.out.println(maxlength);
   }
}
```

```
Run    A04

C:\Users\cse&student\.jdks\openjdk-25.0.2\bin\java.exe "-java
3

Process finished with exit code 0
```

2) Fruit Into Baskets

BRUTE FORCE O(n^2)

```
import java.util.*;
public class A04 {
   public static void main(String args[])
   {
       int arr[]={1,2,3,2,2};
       int max=-1;
       int i,j;
       HashMap<Integer,Integer> h=new HashMap<>();
       for(i=0;i<arr.length;i++)
       {
           h.clear();
           for(j=i;j<arr.length;j++)
           {
               h.put(arr[j],h.getOrDefault(arr[j],0)+1);
               if(h.size()>2)
               {
                   break;
               }
           }
           max=Math.max(max,j-i);
       }
       System.out.println(max);

   }
}
```

```
Run    A04 ×

C:\Users\cse&student\.jdks\openjdk-25.0.2\bin\java.exe "-javaagent:C:
4

Process finished with exit code 0
```

OPTIMAL APPROACH O(n)

```java
import java.util.*;
public class A04 {
   public static void main(String args[])
   {
       int arr[]={1,2,3,2,2};
       int max=-1;
       int i,j=0;
       HashMap<Integer,Integer> h=new HashMap<>();
       for(i=0;i<arr.length;i++)
       {
               h.put(arr[i],h.getOrDefault(arr[i],0)+1);
               while(h.size()>2)
               {
                   h.put(arr[j],h.get(arr[j])-1);
                   if(h.get(arr[j])==0)
                   {
                       h.remove(arr[j]);
                   }
                   j++;
               }

           max=Math.max(max,i-j+1);
       }
       System.out.println(max);

   }
}
```

```
Run    A04

C:\Users\cse&student\.jdks\openjdk-25.0.2\bin\java.exe
4

Process finished with exit code 0
```

3) RANGESUM QUERY

BRUTE FORCE APPROACH

```
class NumArray {
private int[] arr;
public NumArray(int[] nums) {
        arr = nums;
    }
public int sumRange(int left, int right) {
    int sum=0;
    int i=0;
    for(int i=left;i<=right;i++)
    {
        sum=sum+arr[i];
    }
        return sum;
    }
}
```

OPTIMAL APPROACH (PREFIX SUM METHOD)

```
class NumArray {
private int[] prefix;
    public NumArray(int[] nums) {
        prefix = new int[nums.length ];
        prefix[0]=nums[0];
        for (int i = 1; i < nums.length; i++) {
            prefix[i] = prefix[i-1] + nums[i];
        }

    }

    public int sumRange(int left, int right) {
        if(left==0)
        {
            return prefix[right];
        }
        return prefix[right]-prefix[left-1];
    }
```

}

4)Equilibrium Point

Brute Force Method

```
import java.util.*;
public class A04 {
   public static void main(String args[]) {
       int arr[] = {2, 1,0,3};
       int res = -1;
       int i = 0;
       for( i=0;i<arr.length;i++) {
           int left = i-1, right = i+ 1;
           int lsum = 0, rsum = 0;
           while (left >= 0) {
               lsum += arr[left];
               left--;
           }
           while (right < arr.length) {
               rsum += arr[right];
               right++;
           }
           if (lsum == rsum) {
               res = i;
           }

       }
       System.out.println(res);
   }
```

Run A04

```
C:\Users\cse&student\.jdks\openjdk-25.0.2\bin\java.exe
2

Process finished with exit code 0
```

}

Optimal approach

1)

```
class Solution {
    public static int findEquilibrium(int arr[]) {
        int[] prefix=new int[arr.length];
        int[] suffix=new int[arr.length];
        prefix[0]=arr[0];
        for(int i=1;i<arr.length;i++)
        {
            prefix[i]=prefix[i-1]+arr[i];
        }
        suffix[arr.length-1]=arr[arr.length-1];
        for(int i=arr.length-2;i>=0;i--)
        {
            suffix[i]=suffix[i+1]+arr[i];
        }
        for(int i=0;i<arr.length;i++)
        {
            if(prefix[i]==suffix[i])
            {
                return i;
            }
        }
        return -1;
    }
}
```

2)

```
class Solution {
    public static int findEquilibrium(int arr[]) {
```

```
        int[] prefix=new int[arr.length];
        int left=0,right=0;
        prefix[0]=arr[0];
        for(int i=1;i<arr.length;i++)
        {
            prefix[i]=prefix[i-1]+arr[i];
        }
        for(int i=0;i<arr.length;i++)
        {
            left=(i==0)?0:prefix[i-1];
            right=prefix[arr.length-1]-prefix[i];
            if(left==right)
            {
                return i;
            }

        }
        return -1;
    }
}
```

3)

```
import java.util.*;
public class A04 {
   public static void main(String args[]) {
       int arr[] = {-7,1,5,2,-4,3,0};
       int sum1=0,sum2=0;
       for(int i=0;i<arr.length;i++)
       {
           sum1+=arr[i];
       }
       for(int i=0;i<arr.length;i++)
       {

           if(sum2==sum1-arr[i])
           {
                   System.out.println(i);
                   break;
```

```
            }
            sum1-=arr[i];
            sum2+=arr[i];
        }

    }
}
```

```
Run    A04 ×

C:\Users\cse&student\.jdks\openjdk-25.0.2\bin\java.exe "-javaagent:
3

Process finished with exit code 0
```

5) Product of Array Except Self

Brute Force $O(n^2)+O(n)$

```
import java.util.*;
public class A04 {
   public static void main(String args[]) {
       int arr[] = {1,2,3,4};
       int res[]=new int[arr.length];
       int prod;
       for(int i=0;i<arr.length;i++)
       {
           prod=1;
           for(int j=0;j<arr.length;j++)
           {
               if(i!=j)
               {
                   prod*=arr[j];
               }
           }
           res[i]=prod;
       }
           for(int k:res)
```

```
            {
                System.out.print(k+" ");
            }
    }
}
```

```
Run    A04 ×

C:\Users\cse&student\.jdks\openjdk-25.0.2\bin\java.exe "-javaa
24 12 8 6
Process finished with exit code 0
```

Optimal O(n+n+n)

```
import java.util.*;
public class A04 {
    public static void main(String args[]) {
        int arr[] = {1,2,3,4};
        int res[]=new int[arr.length];
        int pre[]=new int[arr.length];
        int suf[]=new int[arr.length];
        pre[0]=1;suf[arr.length-1]=1;
        for(int i=1;i<arr.length;i++)
        {
            pre[i]=pre[i-1]*arr[i-1];
        }
        for(int i=arr.length-2;i>=0;i--)
        {
            suf[i]=suf[i+1]*arr[i+1];
        }

            for(int i=0;i<arr.length;i++)
            {
                res[i]=pre[i]*suf[i];
                System.out.print(res[i]+" ");
            }
    }
```

```
}
```

Run A04

```
C:\Users\cse&student\.jdks\openjdk-25.0.2\bin\j
24 12 8 6
Process finished with exit code 0
```

Most optimal O(n+n)

```
import java.util.*;
public class A04 {
   public static void main(String args[]) {
       int arr[] = {1,2,3,4};
       int res[]=new int[arr.length];
       int pre[]=new int[arr.length];
       pre[0]=1;
       int prod=1;
       for(int i=1;i<arr.length;i++)
       {
           pre[i]=pre[i-1]*arr[i-1];
       }
//        for(int i=arr.length-2;i>=0;i--)
//        {
//            suf[i]=suf[i+1]*arr[i+1];
//        }

           for(int j=arr.length-1;j>=0;j--)
           {
               pre[j]*=prod;
               prod*=arr[j];
           }
// display
```

```
            for(int i:pre)
            {
                System.out.print(i +" ");
            }
    }
}
```

```
Run    A04 ×

C:\Users\cse&student\.jdks\openjdk-25.0.2\bin\java.exe "-javaa
24 12 8 6
Process finished with exit code 0
```